REALCLAWBENCH: Live OpenClaw Benchmarks from Real Developer-Agent Sessions

Anonymous ACL submission

Abstract

Agent benchmarks should reflect what users actually ask deployed agents to do, yet existing benchmarks often miss key realism properties of real developer-agent sessions. We introduce REALCLAWBENCH, a live benchmark framework built from real OpenClaw sessions to capture the distribution, diversity, and real-world difficulty of deployed agent use. Real user requests are challenging to benchmark because they often depend on local execution environments, involve implicit or underspecified intent, and require nontrivial verification. REALCLAWBENCH addresses these challenges with two core mechanisms: reconstructed execution environments and deterministic verifiable scorers, which together convert real sessions into reproducible, automatically scored tasks. The resulting release contains 281 executable tasks sampled from a much larger real-session pool while preserving the source distribution, with maximum final-vs-source Jensen-Shannon divergence of 0.0448. Evaluating 14 contemporary models shows that the best system solves only 65.8% of tasks, revealing substantial headroom on realistic developer-agent workloads. By turning real deployed sessions into controlled evaluation instances, REALCLAWBENCH provides a practical path toward benchmarks that better measure agent capability in actual use. Code is available at: https://anonymous.4open.science/r/real-claw-bench-582B.

1 Introduction

Developer-facing agents are becoming deployed work systems rather than only chat interfaces. The most representative one is OpenClaw[1], which has attracted substantial attention due to its strong practical value, becoming one of the fastest-growing projects on GitHub in terms of stars. In OpenClaw, users ask agents to inspect repositories, call tools, edit files, run commands, and carry out multi-step workflows inside stateful workspaces. This shift changes what evaluation should measure: success should mean completing the user's intended task in the environment where that task arose. Prior work has made agent evaluation more tool-aware, executable, and time-sensitive. ReAct-style agents interleave reasoning with actions (Yao et al., 2022), Toolformer studies API use (Schick et al., 2023), and WebGPT shows evidence gathering in interactive environments (Nakano et al., 2021). SWE-bench evaluates repository-level issue resolution (Jimenez et al., 2024), WebArena studies long-horizon web tasks (Zhou et al., 2024), AgentBench and GAIA broaden evaluation across tools and domains (Liu et al., 2024; Mialon et al., 2024), and LiveBench/LiveCodeBench update public-source tasks to reduce staleness (White et al., 2025; Jain et al., 2025). However, these benchmarks still largely begin from authored tasks, public artifacts, or curated environments rather than deployed developer-agent sessions. As Figure 1 illustrates,

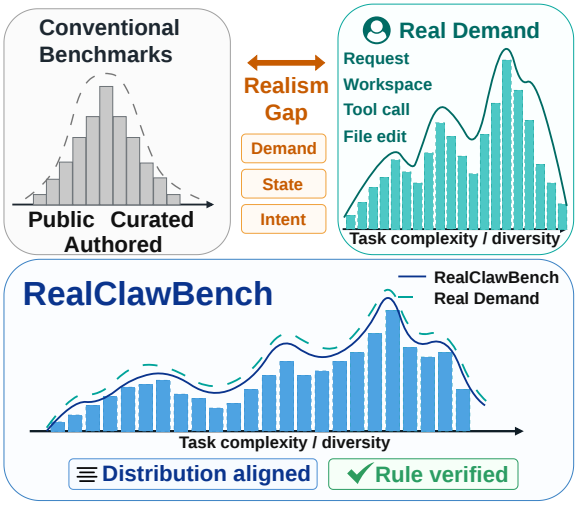

Figure 1: Conceptual overview of the realism gap and the REALCLAWBENCH response.

[1] OpenClaw: https://github.com/openclaw/openclaw

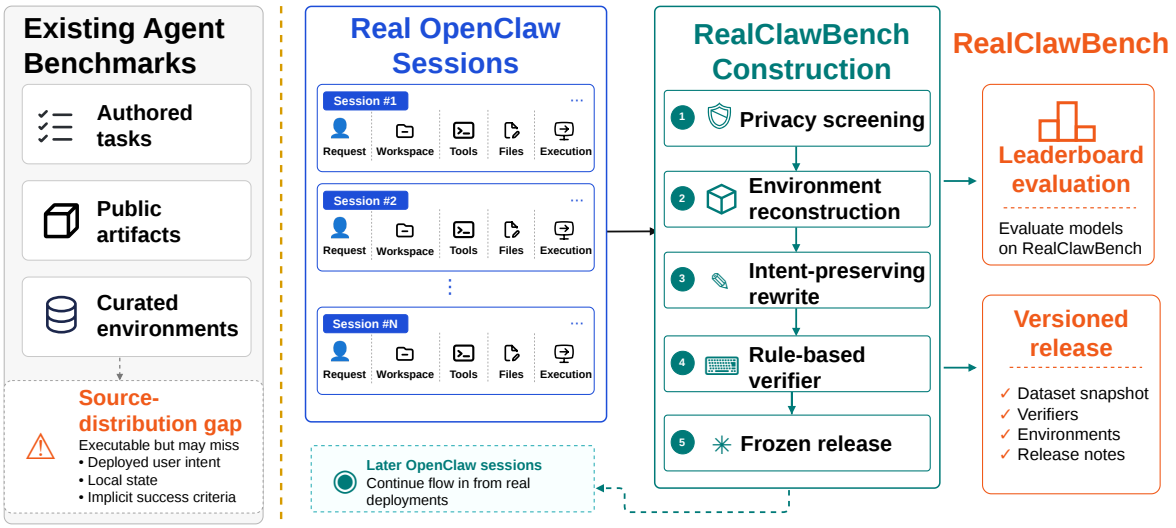

Figure 2: REALCLAWBENCH pipeline from real OpenClaw sessions to frozen releases and live updates. Real sessions define the source distribution, while filtering, sampling, environment reconstruction, request rewriting, and rule-based verification turn that stream into controlled evaluation artifacts.

this creates a realism gap: a benchmark can be executable and difficult while still smoothing away real demand, local workspace state, implicit user intent, and naturally occurring success criteria.

Building a realistic benchmark from deployed sessions is challenging because each task must preserve a rich, recoverable execution environment and support deterministic verification. Real sessions may contain private context, depend on local project state, express intent implicitly, or lack deterministic success criteria. REALCLAWBENCH addresses this challenge by treating real OpenClaw sessions as the source distribution for benchmark construction (OpenClaw, 2026). As Figure 2 shows, the pipeline samples deployed sessions, filters low-quality or unsafe cases, reconstructs execution environments, rewrites requests into standalone instructions, and builds deterministic verifiers. The resulting benchmark therefore measures agents on tasks closer to what users actually experience in deployment. Because the same construction process can be rerun on later sessions, REALCLAWBENCH also supports live releases that reduce staleness and data contamination. This paper makes the following contributions:

- **Realism gap:** We identify the realism gap between executable agent benchmarks and deployed developer-agent use.
- **Session-to-task pipeline:** We present a pipeline that converts real sessions into reproducible, privacy-screened tasks with reconstructed environments and deterministic verifiers.
- **Live benchmark release:** We release REALCLAWBENCH, a live, distribution-anchored benchmark that reveals substantial headroom for current agents on realistic workloads.

2 Related Work

Static and live benchmarks MMLU (Hendrycks et al., 2020), BIG-bench (Srivastava et al., 2023), and HELM (Liang et al., 2022) established broad standardized evaluations for language-model knowledge, reasoning, and reporting discipline. LiveBench (White et al., 2025) and LiveCodeBench (Jain et al., 2025) improve temporal validity by updating items from recent public sources. These benchmarks make model comparison more systematic and less stale, but their items are still collected or authored outside deployed agent workflows. They therefore do not directly measure whether an agent can solve tasks drawn from real sessions with local state, tool traces, and implicit user intent.

Agent benchmarks Agent benchmarks move beyond static question answering by evaluating planning, tool use, and stateful execution. API-Bank (Li et al., 2023) and StableToolBench (Guo et al., 2024) evaluate tool-augmented models and tool-learning stability, while τ-bench adds multi-turn tool-agent-user interaction under structured domain rules (Yao et al., 2024). AgentBench (Liu et al.,

Benchmark	Distribution		User-intent signals				
	JSD↓	TV↓	Intent↑	Local↑	Artifact↑	Constr.↑	Env.↑
RealClawBench	**0.146**	**0.271**	**6.37/8**	**38.4%**	**96.8%**	**91.8%**	**75.4%**
Claw-Eval	0.335	0.538	3.20/8	0.3%	39.7%	19.3%	24.0%
WildClawBench	0.283	0.370	–	–	–	–	–
SWE-bench	0.615	0.804	3.85/8	16.0%	42.3%	42.3%	69.3%
WebArena	1.000	1.000	0.38/8	0.0%	0.0%	0.0%	1.4%
Terminal-Bench 2.0	0.905	0.971	4.73/8	3.4%	51.7%	79.8%	58.4%
WorkArena	0.607	0.798	–	–	–	–	–
OSWorld	1.000	1.000	–	–	–	–	–
AgentBench-OS	–	–	2.28/8	9.7%	0.0%	41.7%	23.6%
GAIA	–	–	0.95/8	0.0%	0.0%	4.5%	18.2%

Table 1: **Realness comparison across agent benchmarks.** Distributional realness is measured against 76,155 deployed OpenClaw tool-use sessions. Lower JSD and TV indicate a closer match to real OpenClaw demand. Text-level user-intent signals are computed from public prompt/task text using transparent rules. RealClawBench is closest to the deployed distribution and retains substantially more local, artifact-oriented, constrained, environment-dependent user-intent signals. Dashes indicate that the public artifact does not support a fair computation for that metric. The detailed computation procedure for the metrics in this table is provided in Appendix H.

2024), WebArena (Zhou et al., 2024), GAIA (Mialon et al., 2024), and SWE-bench (Jimenez et al., 2024) place agents in tool, web, and software environments with executable feedback. OSWorld (Xie et al., 2024) and WorkArena (Drouin et al., 2024) extend the setting to computer-use and enterprise web tasks, and Terminal-Bench evaluates agents on hard, realistic tasks in command-line interfaces (Merrill et al., 2026). SWE-bench Multimodal (Yang et al., 2024) and MLE-bench (Chan et al., 2025) broaden developer-agent evaluation to visual software issues and machine-learning engineering. These benchmarks reveal failures that static tests miss, but their task sources remain largely authored, public-artifact-derived, simulated, or curated. Our focus is complementary: we ask whether the benchmark's source distribution itself matches deployed developer-agent demand.

Real-user data Real-user datasets show why source distributions matter. LMSYS-Chat-1M (Zheng et al., 2024) and WildChat (Zhao et al., 2024) capture large-scale user conversations in the wild. Chatbot Arena (Chiang et al., 2024) and MT-Bench (Zheng et al., 2023) popularized preference-based and judge-based evaluation, while WildBench builds challenging tasks from real-user queries (Lin et al., 2025). These resources capture demand and preference signals that authored prompts often miss. However, they are primarily conversational or output-oriented: they generally do not reconstruct local workspaces, expose the tool-use environment, or verify the final state after an agent acts.

OpenClaw evaluations OpenClaw provides an agent runtime with sessions, tools, workspaces, and harness-level execution surfaces (OpenClaw, 2026). WildClawBench (Ding et al., 2026), Claw-Eval (Ye et al., 2026), and ClawBench (ScoootScooob, 2026) cover OpenClaw-native, full-stack, or trustworthy autonomous-agent settings. OpenClaw safety benchmarks study persistent-state attacks (Wang et al., 2026) and trajectory-level risk (Yang et al., 2026).

Together, these studies show that runtime matters for agent evaluation. REALCLAWBENCH is complementary: it uses the same deployed ecosystem as its starting point, but asks a different question by converting actual OpenClaw user sessions into privacy-screened, executable, and automatically scored benchmark tasks.

3 The Realism Gap in Agent Benchmarks

Executable agent benchmarks have made evaluation more operational, but executability alone does not guarantee realism. A benchmark can require tool use, environment interaction, and executable feedback while still measuring a task distribution that differs from deployed agent use. We therefore compare existing agent benchmarks with the OpenClaw tool-use stream, which contains 76,155 deployed sessions. Table 1 measures this gap from two complementary views: distributional distance from deployed task categories, and user-intent signals tied to local artifacts, explicit constraints, and environment-dependent state.

The comparison shows that existing benchmarks remain separated from deployed developer-agent

use in both distribution and intent signals. Many benchmarks are executable, but they often underrepresent the local, artifact-centered, and state-dependent requests that appear in real sessions. This supports the central premise of REALCLAWBENCH: realistic agent evaluation should not only ask whether an agent can solve controlled tasks, but whether it can solve tasks drawn from the distribution and context in which deployed agents are used. This gap also defines the construction problem: the benchmark must preserve the deployed request distribution while releasing only privacy-screened, self-contained, and automatically scored instances. The next section describes how REALCLAWBENCH constructs such instances.

4 From Real Sessions to Verifiable Tasks

The source data for REALCLAWBENCH come from production OpenClaw usage logs, where real users interact with a deployed developer-agent system for repository inspection, file editing, command execution, data extraction, and project-building workflows. These logs are not researcher-authored prompts: they are user-agent sessions produced during ordinary OpenClaw use, containing user requests, conversation context, available tools, tool calls, workspace traces, and observed outcomes. REALCLAWBENCH converts these deployed sessions into reproducible benchmark tasks. Raw sessions cannot be used directly because they may contain private context, depend on local workspace state, express intent implicitly, or lack deterministic success criteria. Each released instance is constructed through the workflow summarized in Figure 2:

$$b = (x, E_0, V, m),$$

where x is a standalone instruction, E_0 is a sanitized initial workspace, V is an automatic verifier, and m records stratification metadata. We keep only sessions with recoverable intent, reconstructable state, auditable verification, and release-safe content.

4.1 Anchoring the Source Distribution

Using this reference stream, the first stage defines the empirical distribution that the benchmark should preserve. We remove framework-level noise, deduplicate repeated retries of the same task, and retain tool-mediated developer-agent requests. The resulting tool-use stream captures the mix of files, commands, tools, workspace state, and user intents seen in deployment. Importantly, sampling is anchored before final quality filtering. Candidate seeds are drawn from the cleaned tool-use stream rather than from the final high-quality pool, so later filtering cannot silently redefine the target distribution. Common categories follow their empirical mass, while rare categories receive a minimum diagnostic allocation to support fine-grained analysis. Table 2 summarizes the resulting construction funnel, from raw calls to the released evaluation set.

Stage	Name	n	Drop
N_0^A	Raw calls	450,766	–
N_0^S	Sessions	110,170	75.6%
N_0^T	Tool-use	76,155	30.9%
N_1	Cleaned	40,713	46.5%
N_2	HQ pool	6,995	82.8%
N_3	Scorable	5,260	24.8%
C	Candidates	414	92.1%
F	Final	281	32.1%

Table 2: Construction funnel from raw API calls to the released evaluation set. Drop percentages are relative to the previous stage, showing where session folding, tool-use filtering, quality screening, scorer construction, and final review reduce the pool.

4.2 Projecting Sessions into Benchmark Instances

For each sampled seed, we apply three coupled transformations. First, environment reconstruction materializes the initial workspace E_0 from session evidence, including files, dependencies, tests, command context, input data, schemas, and expected output locations. Tasks that depend on private services are fixture-backed when possible and rejected otherwise. Second, request rewriting turns the original user message into a standalone instruction x, resolving local references and conversational ellipsis while removing private identifiers. Rewrites must preserve the task category and must not add solution hints or unavailable evidence. Third, verifier construction builds a deterministic program V over the final workspace, stdout, and bounded subprocesses. Verifiers check structural outcomes such as file existence, schema validity, content matches, command status, or mock-service assertions. Tasks requiring subjective or aesthetic judgment are removed. Figure 3 shows this composition: the inner ring gives the task types, and the outer ring gives the subtasks used for fine-grained analysis.

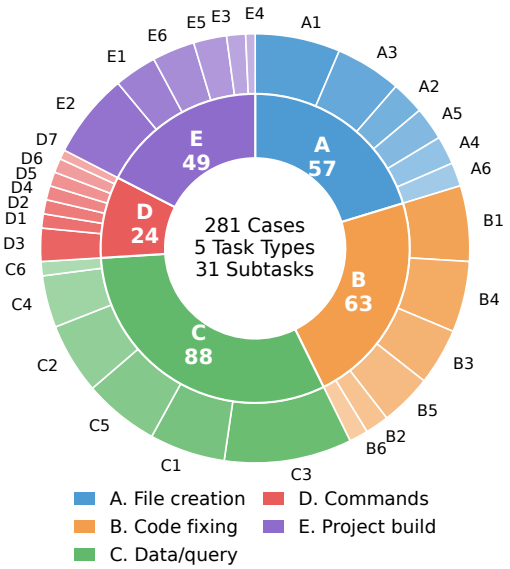

Figure 3: Task composition of the final evaluation set. The inner ring shows the top-level task types, and the outer ring shows subtasks. Full task and subtask names are listed in Appendix C.

4.3 Fidelity Review and Release Filtering

Draft instances are probed under the leaderboard harness to find benchmark bugs, not to score models. We repair missing context, nondeterminism, ambiguous wording, leaky instructions, over-broad verifiers, and environment-free shortcuts only when supported by the original session evidence. Otherwise the instance is rejected. Released tasks must pass reconstructability, rewrite-preservation, verifier-validity, and probing checks. We release only sanitized instructions, workspaces, verifiers, rubrics, metadata, and manifests. Raw logs and private or unreleasable artifacts are excluded.

5 Benchmark Protocol

An evaluated agent receives the standalone instruction x and access to the reconstructed workspace E_0 through a shared OpenClaw harness. For each instance, the harness restores a clean workspace, exposes fixed tools and resource budgets, runs the agent under the same context, turn, timeout, and final-answer protocol, records the trajectory, and invokes the verifier V on the final state.

For model A, the sample-average verifier pass rate is the main leaderboard metric. We also define a distribution-weighted score

$$S_t(A) = \sum_c \hat{p}_t(c) \cdot \frac{1}{|B_{t,c}|} \sum_{b \in B_{t,c}} V_b(A(b)) \quad (1)$$

where $B_{t,c}$ is the set of released instances in category c and $\hat{p}_t(c)$ is the estimated deployed-session mass of category c. Because deployed distributions are imbalanced, the sample score estimates performance on the released set. The weighted score estimates observed-demand performance, while category and subtask scores expose failures on rarer workflows. We also report cost and tool-use statistics. Leaderboard scores are computed only from deterministic verifiers, not LLM judges.

Each release is frozen and versioned by release identifier. Live releases reuse the same construction map T on a later, non-overlapping OpenClaw window and are published as separate versions. Release cards record the log window, filtering counts, sampling weights, category distributions, verifier types, privacy review status, known limitations, and distribution drift. Released artifacts include standalone instructions, sanitized workspaces, verifiers, rubrics, task metadata, and an evaluation manifest. Raw sessions, private identifiers, credentials, proprietary files, and unreleasable service traces are excluded. Tasks depending on private services are fixture-backed or removed.

6 Experiments and Results

We evaluate REALCLAWBENCH along three questions: whether the construction pipeline preserves the distribution of real OpenClaw tool-use sessions, whether the resulting tasks distinguish contemporary agents under a shared executable harness, and whether the same protocol can be reused on later logs for live, versioned releases. Unless otherwise noted, all evaluations use the 281-case release summarized in Table 2 and Figure 3. We run each case with the OpenClaw harness described in Section 5: the harness restores the reconstructed workspace, exposes the same file and command tools, records the agent trajectory, applies the task-specific deterministic verifier, and enforces a 600-second timeout. The headline metric is case-level verifier pass rate. We also report task-type and subtask macro averages to expose long-tail behavior under the naturally imbalanced real-user distribution. For repeated runs, pass@3 denotes the fraction of cases solved in at least one of three runs. Runtime, cost, tool calls, tool failures, and timeouts are diagnostic metrics. All leaderboard scores are computed from deterministic verifiers. The LLM auditor studied in Appendix F is used only for scorer analysis and never for model ranking.

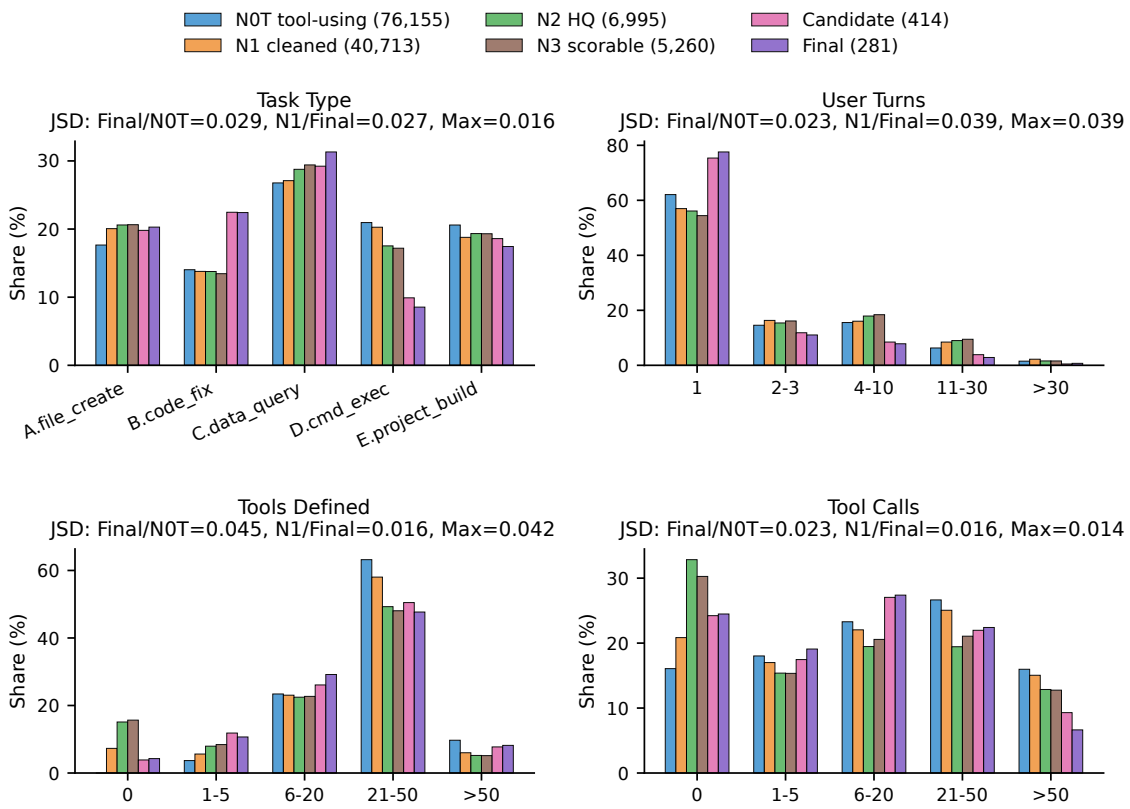

Figure 4: Construction-stage distribution fidelity. Across task type, user turns, tools defined, and tool calls, the final 281-task release remains close to the 76,155-session tool-using reference stream, with final-vs-reference JSD below 0.05 bits on all axes. This shows that filtering, scorer construction, and final review preserve the observable shape of real OpenClaw usage rather than introducing substantial distribution drift.

Comp.	Task	Turns	Tools	Calls
F/N_0^T	0.0294	0.0232	0.0448	0.0233
N_0^T/N_1	0.0009	0.0028	0.0419	0.0027
N_1/N_2	0.0010	0.0009	0.0143	0.0138
N_2/N_3	0.0001	0.0002	0.0001	0.0007
N_3/C	0.0155	0.0392	0.0326	0.0081
C/F	0.0008	0.0010	0.0012	0.0019
N_1/F	0.0265	0.0389	0.0161	0.0162

Table 3: Jensen-Shannon divergence in bits across construction stages and observable distribution axes. The table tests whether filtering and review distort the tool-using reference distribution. All final-vs-reference and adjacent-stage shifts remain below 0.05 bits, indicating that aggressive filtering preserves the observable source distribution.

6.1 Construction Fidelity

This experiment asks whether REALCLAWBENCH can filter aggressively without drifting away from the measured user distribution. The audit uses the construction funnel in Table 2. We use N_0^T as the primary reference because the benchmark targets tool-mediated agent tasks. The full session pool is kept only as a boundary check because it includes many non-agentic requests. The detailed role of each filtering stage is given in Appendix B. Here we evaluate whether the resulting filtering and review process preserves the measured shape of N_0^T. We measure filtering-induced shift against N_0^T along four observable axes: task type, real user turns, tools defined, and tool calls, using Jensen-Shannon divergence (JSD) in bits. Lower values indicate closer distributions.

In Table 3, A/B denotes the comparison between stages A and B, C denotes candidate case cards, and F denotes the final set. For tool-call comparisons, unrecoverable trajectory bins are excluded on both sides. Upstream deterministic labels are mapped into the top-level task types, and the final cases remain traceable to OpenClaw sessions. Appendix C gives the full task table, and Appendix B gives the funnel notes. Figure 4 reports final-vs-reference JSD, direct N_1-to-final JSD, and the largest adjacent-stage drift in its panel subtitles. Table 3 and Figure 4 show that final-vs-reference JSD stays below 0.05 bits on all four axes: 0.0294 for

Rank	Model	Sample	Task						Subtask	pass@3
			file creation	code fixing	data/codebase querying	command execution	project building	Avg.		
1	Claude Opus 4.7	**65.8%**	66.1%	*58.7%*	**68.6%**	65.3%	**70.1%**	**65.7%**	**65.9%**	75.1%
2	GPT-5.5	65.0%	**70.2%**	57.7%	64.4%	65.3%	69.4%	65.4%	65.8%	75.1%
3	MiMo V2.5 Pro	*60.1%*	54.4%	**61.4%**	59.8%	*63.9%*	*63.9%*	*60.7%*	*63.0%*	*74.7%*
4	DeepSeek V4 Pro	59.8%	52.6%	60.3%	*61.7%*	*63.9%*	61.9%	60.1%	61.5%	**76.2%**
5	GLM 5.1	57.4%	53.8%	51.9%	58.7%	**66.7%**	61.9%	58.6%	60.3%	70.8%
6	Kimi K2.6	57.7%	59.1%	56.1%	58.3%	**66.7%**	52.4%	58.5%	60.2%	73.0%
7	Gemini 3.1 Pro	57.4%	55.0%	52.4%	60.2%	59.7%	60.5%	57.6%	58.9%	69.0%
8	DeepSeek V4 Flash	56.0%	60.2%	50.3%	55.7%	62.5%	55.8%	56.9%	58.1%	71.2%
9	Qwen 3.6 Plus	55.8%	*61.4%*	47.6%	56.4%	54.2%	59.2%	55.8%	56.1%	67.6%
10	Claude Sonnet 4.6	55.3%	56.1%	49.7%	59.1%	52.8%	55.8%	54.7%	55.5%	70.5%
11	MiniMax M2.7	53.7%	53.8%	49.2%	57.2%	55.6%	52.4%	53.6%	53.7%	68.3%
12	Claude Opus 4.6	52.1%	46.8%	50.3%	51.5%	58.3%	58.5%	53.1%	52.6%	67.3%
13	Gemma 4 31B	45.7%	25.1%	47.6%	56.1%	52.8%	44.9%	45.3%	47.5%	51.6%
14	GPT-OSS 120B	42.7%	35.7%	47.6%	42.4%	56.9%	38.1%	44.2%	43.1%	57.7%

Table 4: Main leaderboard on the REALCLAWBENCH evaluation set, including average scores, task-type scores, subtask macro-average, and pass@3. Under Task, Avg. is the macro-average over the five task types. First, second, and third places in each metric column are marked with bold, underline, and italics, respectively.

task type, 0.0232 for user turns, 0.0448 for tools defined, and 0.0233 for tool-call count. All adjacent-stage comparisons are also below 0.05 bits, including candidate-to-final manual review, and the direct N_1-to-final comparison remains below 0.039 bits on every axis. Thus REALCLAWBENCH remains selective without losing the measured shape of N_0^T.

6.2 Main Leaderboard

We next test whether the reconstructed benchmark separates current agents under a shared execution protocol. Table 4 reports the 14-model leaderboard on the released REALCLAWBENCH benchmark. The evaluated systems span proprietary frontier APIs and open-weight or openly documented models from major model families. They include Claude Opus/Sonnet variants (Anthropic, 2026), OpenAI GPT-5.5 (OpenAI, 2026) and GPT-OSS (OpenAI, 2025), Google Gemini (Google DeepMind, 2026) and Gemma (Google, 2026), and MiMo (Xiaomi, 2026), DeepSeek (DeepSeek-AI, 2026), GLM (Zhipu AI, 2026), Kimi (Moonshot AI, 2026), Qwen (Alibaba, 2026), and MiniMax models (MiniMax AI, 2026). Sample is the case-level micro-average. Under Task, the five named columns report task-type pass rates, and Avg. is their macro-average. Subtask is the 31-subtask macro-average. The table averages these quantities over three independent runs, and pass@3 reports the fraction of cases solved at least once. This repeated-run metric distinguishes average performance from occasional success.

Overall results Claude Opus 4.7 is the strongest model across aggregate views, reaching 65.8% sample-average success and leaving roughly one third of the benchmark unsolved. MiMo V2.5 Pro ranks third by sample average but remains competitive under macro-averaging, with a 63.0% subtask average. Models such as GLM 5.1 and Kimi K2.6 are relatively stronger on macro-averaged views than on the raw sample average, showing why long-tail reporting matters for imbalanced releases. The gap between Sample and pass@3 further shows that several agents can solve additional cases across attempts but do not solve them reliably.

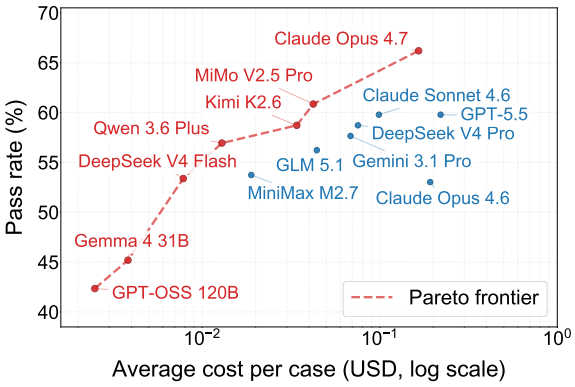

Figure 5: Accuracy-cost tradeoff between sample-average pass rate and per-case cost. The figure shows that higher spending does not directly translate into higher accuracy, and that several mid-cost models offer competitive frontier points.

Efficiency and cost Figure 5 shows that performance and operational cost are not tightly aligned.

Each point is one model. The x-axis is average cost per case on a log scale, the y-axis is sample-average pass rate, and the dashed red line marks the cost-performance frontier. Frontier models are labeled in red, while the remaining models are labeled in blue. Claude Opus 4.7 costs $46.54 for a full benchmark run, while GPT-5.5 costs $61.87 but ranks lower. MiMo V2.5 Pro offers a favorable top-tier tradeoff at $11.87. The two cheapest models, Gemma 4 31B and GPT-OSS 120B, cost $1.35 and $2.04 but trail the top models by more than 20 sample-average points. Together, the leaderboard, macro-averages, and cost results show that model rank, long-tail robustness, and operational efficiency should be reported jointly. Appendix C gives the full subtask robustness matrix and shows where the long-tail ordering differs from the headline leaderboard.

6.3 Live Benchmarking on Later Logs

We finally test live benchmarking by rerunning the construction pipeline on two non-overlapping temporal windows: a base window W_{base} and a later live window W_{live}. No task labels, quality predicates, rewriting rules, or scorer-construction procedures are changed. This design tests whether the documented construction procedure remains reusable, rather than whether the same user requests recur. The filtering, scorer-constructability, and final-selection stages are then applied to W_{live}.

Stage	W_{base}	W_{live}
N_0^T Tool-use	76,155	11,638
N_1 Cleaned	40,713	3,505
N_2 HQ pool	6,995	606
N_3 Scorable	5,260	512
Final	281	34

Table 5: Pipeline counts for W_{base} and W_{live} under the same construction protocol. The live window still yields scorer-constructable and final cases, showing that the pipeline remains usable on later logs without redesigning the filters. Final denotes cases that remain after the same final-selection stage.

Live yield. Table 5 shows that the pipeline remains quantitatively productive on the later window. W_{live} yields 11,638 tool-use sessions, 3,505 cleaned cases, 606 high-quality cases, 512 scorer-constructable cases, and 34 final cases. The scorer-constructable retention ratio over N_1 is 14.6% in W_{live}, comparable to 12.9% in W_{base}, supporting the live-yield claim.

Distribution drift. Table 6 measures drift along the same four axes as Experiment 1, with all Jensen-Shannon divergence values reported in bits. B_i denotes stage N_i in W_{base}, L_i denotes stage N_i in W_{live}, and F denotes the final release. The live window remains distributionally close to the base window: the largest W_{live}-vs.-W_{base} divergence is 0.0217 at the cleaned stage and 0.0411 at the scorer-constructable stage. The current final set is also close to both scorer-constructable pools, with all reported JSD values below 0.061 bits. Appendix D visualizes the normalized retention and distribution-drift patterns. Thus the later stream is comparable but not identical, motivating versioned live releases. REALCLAWBENCH can keep fixed releases for controlled leaderboards and use later versions to track deployed demand.

Axis	L_1/B_1	L_3/B_3	F/B_3	F/L_3
Task	0.0135	0.0210	0.0186	0.0370
Turns	0.0099	0.0195	0.0485	0.0609
Tools	0.0195	0.0411	0.0410	0.0315
Calls	0.0217	0.0244	0.0227	0.0095

Table 6: Temporal Jensen-Shannon divergence between W_{base}, W_{live}, and final-release distributions. All reported values remain small, indicating that the live window is comparable to the base release while still reflecting temporal demand shift.

7 Conclusion

We presented REALCLAWBENCH, an empirically grounded live agent benchmark framework built from real OpenClaw user sessions. Rather than manually authoring tasks, the pipeline projects deployed requests into privacy-screened instructions, reconstructed workspaces, and programmatic scorers. The construction experiment shows that substantial filtering and review can preserve the observable shape of the tool-using real-user distribution. The main leaderboard evaluates 14 models and shows that the best model reaches 65.8% sample-average success, leaving substantial headroom on real developer-agent tasks. The temporal live experiment further produces 512 scorer-constructable cases and 34 final cases from a later log window under the same documented protocol. Together, these results support a benchmark design in which deployed usage provides the source distribution, while reconstruction, scoring, and versioned live releases turn a changing request stream into reproducible controlled evaluation.

Limitations

This manuscript reports construction fidelity, the main leaderboard, live evaluation, and scorer validation in the appendix. Several limitations remain. First, the benchmark reflects OpenClaw's developer-oriented user population, product surface, tool interface, and logging policy, so it should not be read as a universal estimate of all agent workloads. Second, filtering and reconstruction can still shift the distribution away from raw usage. Tasks depending on private services, unreleasable files, credentials, GUI state, or unstable external resources are more likely to be removed or simplified. Third, programmatic scorers favor outcomes that can be checked in a final workspace or bounded execution trace. They may miss qualities such as code maintainability, design quality, security posture, and partial but useful progress. Fourth, model scores are conditioned on one harness, timeout, tool interface, and cost model. Different budgets or tool implementations could change absolute scores, even if the construction protocol remains fixed. Finally, live releases require governance so that privacy review, versioning, scorer audits, and leaderboard comparability remain reliable as the data stream changes.

Ethical Considerations

REALCLAWBENCH begins from sensitive real user sessions, so responsible data handling is central to the benchmark design. The pipeline should use only data collected under appropriate consent, retention, and access-control policies. Raw logs should remain internal and should not be released as benchmark items. Before inclusion, sessions must be systematically screened for personal data, credentials, proprietary secrets, harmful content, and licensing constraints. Released environments should contain only sanitized artifacts required for evaluation, with private identifiers removed and private service dependencies replaced by local fixtures when faithful reconstruction is possible. When a task cannot be safely reconstructed, it should be excluded rather than manually paraphrased into a misleading public benchmark item.

The benchmark also affects model incentives. Because the task distribution is drawn from real usage, poor benchmark design could encourage models to optimize for common requests while neglecting rare but consequential failures. It could also encourage models to exploit brittle verifier shortcuts rather than complete the intended task. For this reason, REALCLAWBENCH reports category-level scores, audits scorer behavior, and documents filtering decisions for each release.

References

Alibaba. 2026. Qwen 3.6 Plus. https://qwen.ai/blog?id=qwen3.6. Accessed: 2026-05-24.

Anthropic. 2026. Claude models overview. https://platform.claude.com/docs/claude/docs/models-overview. Accessed: 2026-05-24.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, and 1 others. 2025. Mle-bench: Evaluating machine learning agents on machine learning engineering.

Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E Gonzalez, and 1 others. 2024. Chatbot arena: An open platform for evaluating llms by human preference.

DeepSeek-AI. 2026. DeepSeek-V4-Pro and DeepSeek-V4-Flash. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro. Accessed: 2026-05-24.

Shuangrui Ding, Xuanlang Dai, Long Xing, Shengyuan Ding, Ziyu Liu, Yang JingYi, Penghui Yang, Zhixiong Zhang, Xilin Wei, Xinyu Fang, and 1 others. 2026. Wildclawbench: A benchmark for real-world, long-horizon agent evaluation.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, and 1 others. 2024. Workarena: How capable are web agents at solving common knowledge work tasks?

Google. 2026. Gemma 4 31B model card. https://huggingface.co/google/gemma-4-31B. Accessed: 2026-05-24.

Google DeepMind. 2026. Gemini 3.1 Pro model card. https://deepmind.google/models/model-cards/gemini-3-1-pro/. Accessed: 2026-05-24.

Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*.

Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, pages 58791–58831.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. Api-bank: A comprehensive benchmark for tool-augmented llms.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, and 1 others. 2022. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*.

Bill Yuchen Lin, Yuntian Deng, Khyathi Chandu, Abhilasha Ravichander, Valentina Pyatkin, Nouha Dziri, Ronan Le Bras, and Yejin Choi. 2025. Wildbench: Benchmarking llms with challenging tasks from real users in the wild.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, and 1 others. 2024. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, and 66 others. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *Preprint*, arXiv:2601.11868.

Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. Gaia: a benchmark for general ai assistants. In *International Conference on Learning Representations*, volume 2024, pages 9025–9049.

MiniMax AI. 2026. MiniMax M2.7. https://huggingface.co/MiniMaxAI/MiniMax-M2.7. Accessed: 2026-05-24.

Moonshot AI. 2026. Kimi K2.6. https://platform.kimi.ai/docs/models. Accessed: 2026-05-24.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, and 1 others. 2021. Webgpt: Browser-assisted question-answering with human feedback.

OpenAI. 2025. gpt-oss-120b and gpt-oss-20b model card. https://openai.com/index/gpt-oss-model-card/. Accessed: 2026-05-24.

OpenAI. 2026. Introducing GPT-5.5. https://openai.com/index/introducing-gpt-5-5/. Accessed: 2026-05-24.

OpenClaw. 2026. OpenClaw agent runtimes. https://docs.openclaw.ai/concepts/agent-runtimes. Accessed: 2026-05-24.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551.

ScoootScooob. 2026. Clawbench: Trace-scored agent benchmark with dynamical-systems diagnostics.

Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, and 1 others. 2023. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on machine learning research*.

Zijun Wang, Haoqin Tu, Letian Zhang, Hardy Chen, Juncheng Wu, Xiangyan Liu, Zhenlong Yuan, Tianyu Pang, Michael Qizhe Shieh, Fengze Liu, and 1 others. 2026. Your agent, their asset: A real-world safety analysis of openclaw.

Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Benjamin Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, and 1 others. 2025. Livebench: A challenging, contamination-limited llm benchmark.

Xiaomi. 2026. MiMo V2.5 Pro. https://mimo.xiaomi.com/. Accessed: 2026-05-24.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, and 1 others. 2024. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments.

John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, and 1 others. 2024. Swe-bench multimodal: Do ai systems generalize to visual software domains?

Zhonghao Yang, Yu Li, Yanxu Zhu, Tianyi Zhou, Yuejin Xie, Haoyu Luo, Jing Shao, Xia Hu, and Dongrui Liu. 2026. Benchmarks for trajectory safety evaluation and diagnosis in openclaw and codex: Atbench-claw and atbench-codex.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A benchmark for tool-agent-user interaction in real-world domains.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, Qi Liu, Zhifang Sui, and Tong Yang. 2026. Claw-eval: Towards trustworthy evaluation of autonomous agents. *Preprint*, arXiv:2604.06132.

Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. 2024. Wildchat: 1m chatgpt interaction logs in the wild.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric Xing, and 1 others. 2024. Lmsys-chat-1m: A large-scale real-world llm conversation dataset. In *International Conference on Learning Representations*, volume 2024, pages 22225–22257.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, and 1 others. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623.

Zhipu AI. 2026. GLM-5.1. https://glm5.ai/. Accessed: 2026-05-24.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, and 1 others. 2024. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pages 15585–15606.

A Experimental Configuration

Table 7 summarizes the main settings used in our experiments. We include the benchmark size, execution environment, model-serving setup, timeout limits, and sampling policy. To preserve anonymity, the table omits local file paths, private service identifiers, and other deployment-specific details that are not needed for reproducing the reported experimental protocol. These settings should be read as fixed control variables for the leaderboard: each model receives the same workspace reset, tool interface, timeout, and sampling configuration. The hardware rows describe the shared evaluation infrastructure rather than model-specific resources. The table is therefore intended to make the run conditions auditable, not to define a new experimental variable. Cost estimates in the main text are computed from recorded input and output token counts and the model-specific prices used at evaluation time. For locally served open-weight models, the reported costs should be interpreted as evaluation-accounting estimates rather than provider billing receipts.

Item	Configuration
Benchmark size	281 tasks, 14 models
Runtime	OpenClaw agent runtime
Workspace	One isolated workspace per task
Scoring	Case-specific Python verifiers
Allowed tools	File: read, write, edit Search: file search, file list Execution: shell command
CPU machines	20 machines, 32 vCPUs each
Controller CPU	2 × AMD EPYC 7742, 256 logical cores
Controller memory	2.0 TiB RAM
GPU server	8 × NVIDIA A100-SXM4-80GB
Operating system	Ubuntu 24.04
Python version	Python 3.12
Node.js version	Node.js 22.x in Docker
Local model server	vLLM 0.19.1
Task timeout	600 s
Sampling	temperature = 1.0 top_p = 1.0 top_k = -1 min_p = 0.0

Table 7: Experimental configuration for the main evaluation.

B Construction Funnel Details

Table 8 expands the compact funnel in Table 2 by recording each stage's role and main filtering reason. N_0^A and N_0^S are log-accounting stages, while N_0^T defines the tool-using reference population

Stage	Description	Main Reason
N_0^A	Raw API calls	Source OpenClaw API-call logs.
N_0^S	Unique user sessions	Fold by 5-turn-prefix hash. Remove sessions whose user turns are all framework boilerplate.
N_0^T	Tool-using sessions	Keep sessions with at least one defined tool. This is the primary real-user reference for agent benchmarking.
N_1	Framework-cleaned	Remove framework injection, system templates, memory jobs, cron messages, subagent dispatches, and related non-user turns.
N_2	High-quality pool	Enforce request length, intent signal, no secrets, within-label deduplication, and removal of continuation, meta, and greeting-only sessions.
N_3	Scorer-constructable	Drop failed or unclear sessions without a verifiable final state.
C	Candidate case cards	Construct task, rubric, seed workspace, and verifier. Exclude aggregate markdown files.
F	Final evaluation cases	Manual review removes ambiguous, unstable, unreproducible, or low-quality cases.

Table 8: Construction-funnel details with stage descriptions and main filtering reasons. The table clarifies what each stage removes and why the large reductions correspond to reproducibility, privacy, quality, and scorer-constructability requirements.

used in the construction-fidelity experiment. The later stages are benchmark-construction gates: N_1 removes framework artifacts, N_2 keeps sessions with sufficient intent signal, N_3 requires plausible automatic verification, and C and F turn records into reviewed benchmark artifacts. The large drops therefore reflect the cost of turning private, stateful production sessions into public, reproducible, and scorable tasks. The key point is that most filtering is tied to reproducibility and verification requirements rather than to late manual rebalancing. Future releases can report the same funnel so observed changes can be attributed to user-demand drift or explicit construction-policy changes.

C Task and Subtask Distribution

Table 9 reports the complete task and subtask composition of the final evaluation set. Share of all is computed over the full final set, and share of task is computed within the corresponding top-level task type. The task-type rows correspond to the five inner-ring categories in Figure 3, while the subtask rows correspond to the outer-ring slices. The taxonomy is operational rather than purely semantic: top-level task types describe what the agent must do in the environment, and subtasks provide finer labels for sampling, macro-averaging, and error analysis. These labels are metadata only and are not shown to the evaluated agent. The distribution is visibly imbalanced, which is expected for a benchmark distilled from real usage. The small subtasks should therefore be read as diagnostic slices rather than independently powered benchmarks. `C.data_query` is the largest group, accounting for 31.3% of the release, while `D.cmd_exec` is the smallest top-level group at 8.5%. Several subtasks contain only two to five cases, which is why the main results report both sample-average pass rate and macro-averages over task types and subtasks. The task-type columns in the leaderboard correspond to the five top-level task rows, while the subtask matrix expands them into the 31 outer-ring labels. The three largest top-level categories, data/codebase querying, code fixing, and file creation, account for 74.0% of the final set, so a pure sample average would be strongly shaped by these workflows. At the subtask level, the largest slice is behavior or logic analysis with 27 cases, while multiple command-execution and project-building subtasks contain only two or three cases. Because the benchmark is not manually balanced, the sample average estimates performance under the released distribution, the five-way task average reduces dominance by the largest top-level category, and the 31-way subtask average highlights long-tail robustness. Figure 6 reports the full model-by-subtask matrix used for this analysis. Its first column is the macro-average over all subtasks, while the remaining columns show pass rates for each subtask separately. This view is larger than the main leaderboard, so we place it in the appendix, but it is important for interpreting close model comparisons. The ordering is close to the main table but not identical: MiMo V2.5 Pro moves closer to Claude Opus 4.7 under subtask macro-averaging, and several mid-table comparisons shift when rare subtasks receive equal weight. The matrix also makes clear that high overall performance can hide uneven behavior across task families. For example, a model can perform well on frequent data-querying cases while losing ground on smaller

Task/Subtask	Description	Cases	Share of all	Share of task
A.file_create	File creation	57	20.3%	–
A1.single_html_game	Single-file HTML game	18	6.4%	31.6%
A3.code_module	Code module or script	14	5.0%	24.6%
A2.web_app_page	Web app page or courseware	7	2.5%	12.3%
A5.doc_report	Document or report	7	2.5%	12.3%
A4.manim_video_script	Animation or video script	6	2.1%	10.5%
A6.binary_data_file	Data or office file	5	1.8%	8.8%
B.code_fix	Code fixing	63	22.4%	–
B1.bug_fix	Runtime or compile bug fix	16	5.7%	25.4%
B4.refactor	Refactoring and code quality	15	5.3%	23.8%
B3.feature_tweak	Small feature adjustment	12	4.3%	19.0%
B5.feature_add	New feature or module	11	3.9%	17.5%
B2.config_fix	Configuration fix	5	1.8%	7.9%
B6.review_report	Code or document review	4	1.4%	6.3%
C.data_query	Data and codebase querying	88	31.3%	–
C3.behavior_logic_analysis	Behavior or logic analysis	27	9.6%	30.7%
C1.locate_specific_code	Locate specific code or references	16	5.7%	18.2%
C5.extract_structured_data	Extract structured data	16	5.7%	18.2%
C2.codebase_inventory_report	Codebase inventory report	15	5.3%	17.0%
C4.print_file_contents	Print file contents	11	3.9%	12.5%
C6.system_env_inspection	System or environment inspection	3	1.1%	3.4%
D.cmd_exec	Command execution	24	8.5%	–
D3.write_script_automation	Write automation script	7	2.5%	29.2%
D1.git_clone_inspect	Git clone and inspect changes	3	1.1%	12.5%
D2.fs_inventory_analyze	Filesystem inventory and analysis	3	1.1%	12.5%
D4.create_files_config	Create files or edit config	3	1.1%	12.5%
D5.install_setup_env	Package installation or environment setup	3	1.1%	12.5%
D6.diagnose_cleanup	Diagnosis and cleanup	3	1.1%	12.5%
D7.doc_convert_audit	Document conversion or audit	2	0.7%	8.3%
E.project_build	Project building	49	17.4%	–
E2.module_feature_impl	Module or feature implementation	18	6.4%	36.7%
E1.web_app_prototype	Web-app prototype or game	9	3.2%	18.4%
E6.spec_doc_update	Specification or documentation update	9	3.2%	18.4%
E5.audit_review_report	Code or policy audit report	7	2.5%	14.3%
E3.skill_plugin_pack	Skill or plugin packaging	4	1.4%	8.2%
E4.scaffold_setup	Project scaffolding setup	2	0.7%	4.1%
TOTAL	Total	281	100.0%	–

Table 9: Complete task and subtask distribution for the final evaluation set, including within-task and overall shares. The table makes the real-usage imbalance explicit and explains the task and subtask labels used for macro-averaged reporting.

creation, configuration, or command-execution subtasks. This supports reporting sample, task, and subtask averages together rather than treating the micro-average as the only benchmark summary.

D Live Benchmark Diagnostics

The main text reports the live-window counts and JSD values in compact tables. Here we include the corresponding visual diagnostics for the same temporal split. Figure 7 normalizes each window by its N_1 cleaned count and reports the retained share at the N_2 high-quality and N_3 scorer-constructable stages. This view makes the yield comparison easier to read: although W_{live} is smaller in absolute size, its scorer-constructable retention remains comparable to W_{base}. Figure 8 complements Table 6 by showing the same drift comparisons across task type, user turns, tools defined, and tool calls. In normalized terms, W_{live} keeps 17.3% of cleaned cases at N_2 and 14.6% at N_3, close to 17.2% and 12.9% in W_{base}. Thus the later window is smaller mainly because its log span is shorter, not because the construction pipeline stops finding high-quality or scorable sessions. The bars compare the base and live windows at N_1 and N_3 together with the current final release. Across all four axes, the distributions remain close enough for cross-release comparison while still showing measurable temporal variation. The largest base-live drift is 0.0411 bits for tools defined at N_3, and the final set remains within 0.061 bits of the live-window scorer-constructable pool on all reported axes.

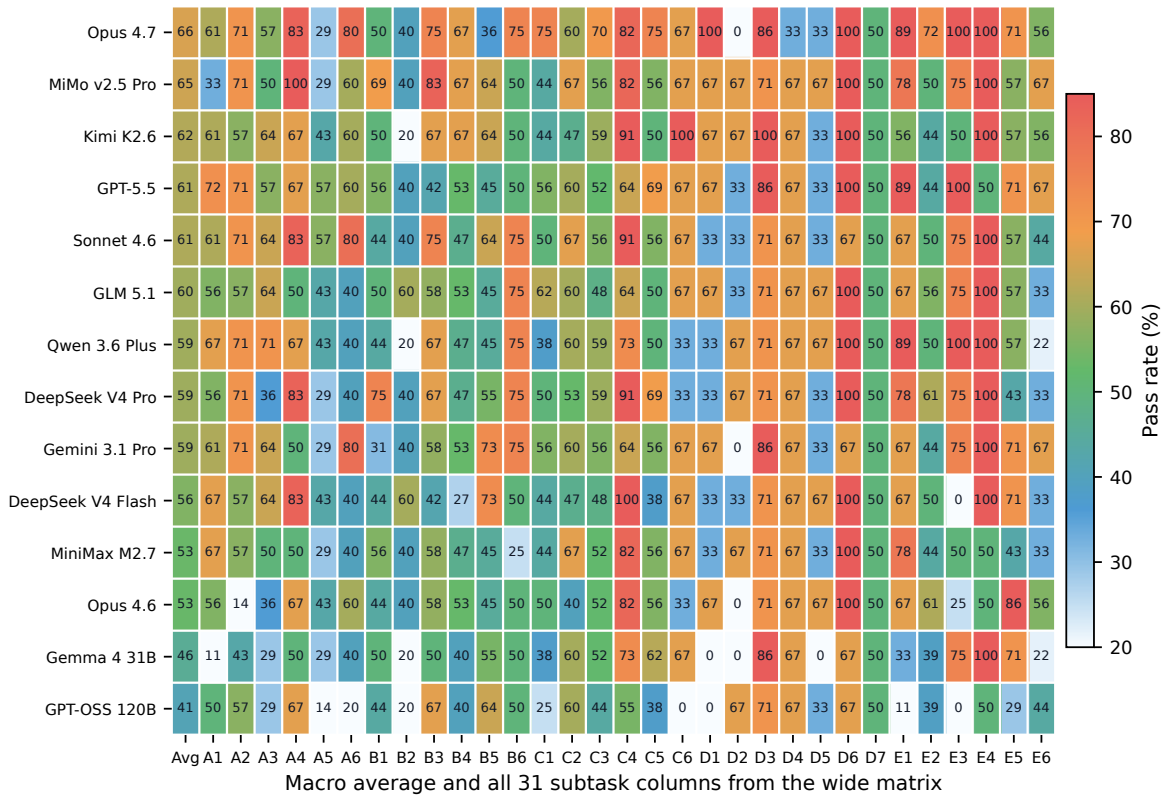

Figure 6: Subtask robustness matrix showing model pass rates across the full subtask taxonomy. The matrix reveals that rankings can shift under long-tail macro-averaging, so headline sample accuracy alone is not sufficient.

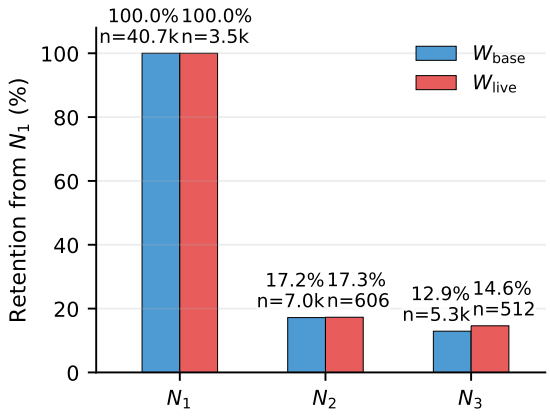

Figure 7: Live-window retention after normalizing each temporal window by its N_1 cleaned count. The live window retains a comparable scorer-constructable share to the base window, supporting the live-yield claim.

Rank	Model	Sample	Task	pass@3
1	GPT-5.5	**71.6%**	**75.3%**	**73.5%**
2	Claude Opus 4.7	61.8%	67.9%	70.6%
3	Kimi K2.6	61.8%	61.2%	73.5%
4	Claude Sonnet 4.6	60.8%	57.8%	73.5%
5	Claude Opus 4.6	59.8%	60.5%	70.6%
6	Gemini 3.1 Pro	58.8%	63.7%	70.6%
7	MiMo V2.5 Pro	58.8%	58.5%	76.5%
8	GLM 5.1	57.8%	59.9%	70.6%
9	DeepSeek V4 Pro	56.9%	55.4%	64.7%
10	Qwen 3.6 Plus	54.9%	54.5%	64.7%
11	MiniMax M2.7	52.9%	52.8%	61.8%
12	DeepSeek V4 Flash	52.0%	51.4%	61.8%
13	Gemma 4 31B	49.0%	47.1%	55.9%
14	GPT-OSS 120B	44.1%	45.8%	61.8%

Table 10: Leaderboard on the 34-case extra-quality live subset. The subset acts as a live sanity check and continues to separate models, with GPT-5.5 leading on both Sample and Task.

E Extra-Quality Live Subset

The 34 final cases from W_{live} provide a compact extra-quality subset for checking whether the same evaluation protocol continues to separate models on newly constructed tasks. Table 10 reports the corresponding leaderboard.

Sample follows the same micro-average definition as the main table, Task is the task-type macro average, and pass@3 reports the fraction of cases solved at least once. GPT-5.5 leads this subset on both Sample and Task, while Claude Opus 4.7 and Kimi K2.6 form a close second tier with identical Sample but different macro profiles. MiMo V2.5 Pro has the highest pass@3, suggesting that

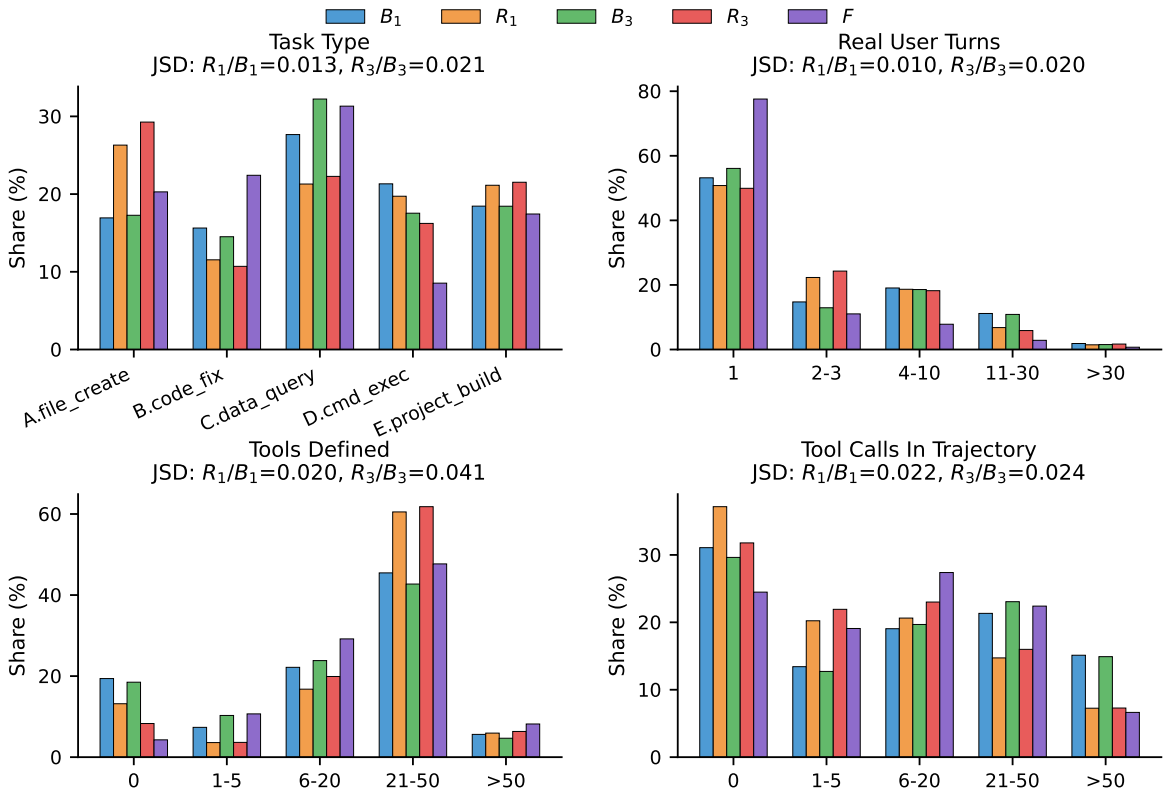

Figure 8: Distribution drift under live benchmarking. The figure tests whether the later window remains comparable to the base window across task type, user turns, tools defined, and tool calls. The distributions remain close across all four axes, showing that the live stream preserves the benchmark's source distribution while still capturing temporal variation.

some failures are recoverable across repeated runs. The spread between the top and bottom models is 27.5 Sample points, so even this small live subset remains discriminative. The gap between Sample and pass@3 is also informative: MiMo V2.5 Pro solves at least one run on 76.5% of cases but averages 58.8%, indicating many non-repeatable successes. Because this subset is small, we use it as a live sanity check rather than a replacement for the main leaderboard.

F Environment and Scorer Validation

The main leaderboard uses case-specific rule-based programmatic verifiers rather than LLM judges. This appendix asks a separate validation question: whether an LLM auditor can be reliable when it is given execution evidence. The goal is to test the value of execution evidence for LLM auditing, not to replace the rule-based verifiers used for model ranking. We compare human judgment with the RealClawBench full-evidence LLM auditor and an output-only LLM judge baseline on the final benchmark. The output-only baseline receives only the task, rubric, and final model output.

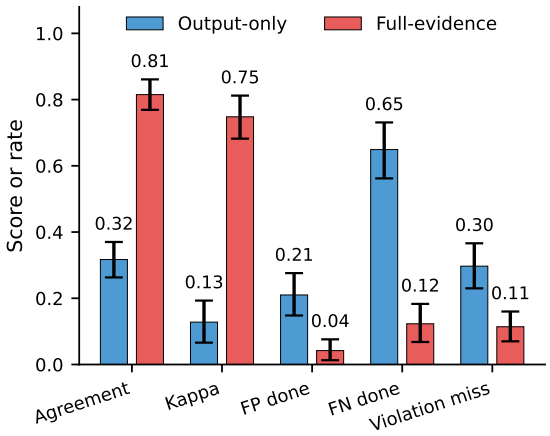

Figure 9: Scorer-validation metrics comparing agreement and benchmark-critical error rates. Full-evidence auditing improves agreement with human labels while reducing false-positive done, false-negative done, and violation-miss errors.

The full-evidence LLM auditor additionally sees verifier checks, tool summaries, runtime metadata,

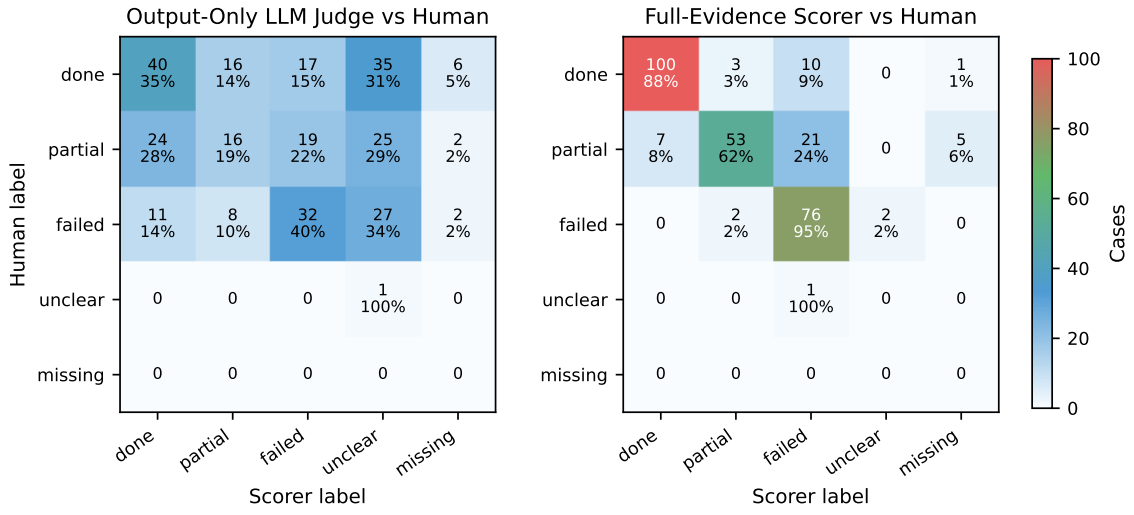

Figure 10: Verdict confusion matrices with human labels as rows and scorer labels as columns. The full-evidence auditor is more concentrated on the diagonal, while output-only judging more often confuses done, partial, failed, and unclear outcomes.

and reconstructed task evidence. This matters because many agent outcomes are invisible from the final answer alone: an agent may claim edits it never made, ignore required evidence, or fail after a command error while still producing a confident response. The human reference separates successful completion, partial progress, failed attempts, unclear outcomes, and missing outputs. We report exact-verdict agreement and Cohen's κ for overall alignment, plus false-positive done, false-negative done, and violation-miss rates for benchmark-critical error modes. Lower is better for the three error rates. Figure 9 first summarizes the main pattern: the full-evidence LLM auditor improves agreement with human labels while reducing all three failure modes, rather than merely becoming more permissive.

Metric	Output-only	Full-evidence
Compared cases	281	281
Exact-verdict agreement	0.317	**0.815**
Cohen's κ	0.128	**0.748**
False-positive done rate	0.210	**0.042**
False-negative done rate	0.649	**0.123**
Violation miss rate	0.297	**0.114**

Table 11: Scorer-validation summary against human labels for output-only and full-evidence LLM auditing. Full-evidence auditing achieves much higher agreement and lower done-label error rates, showing the value of execution evidence.

Table 11 then gives the exact values. The full-evidence LLM auditor reaches 81.5% exact-verdict agreement and $\kappa = 0.748$, compared with 31.7% and $\kappa = 0.128$ for the output-only baseline. It also reduces false-positive done from 21.0% to 4.2% and false-negative done from 64.9% to 12.3%, supporting execution-aware LLM auditing for stateful agent tasks. In absolute terms, execution evidence improves exact agreement by 49.8 percentage points and raises Cohen's κ by 0.620. The error reductions are benchmark-critical: false-positive done labels fall by a factor of five, and false-negative done labels fall by more than fivefold, which means the auditor is less likely to both over-credit and under-credit completed agent work. Figure 10 gives the label-level view. Rows are human labels and columns are auditor labels. The full-evidence LLM auditor is much more concentrated on the diagonal, while the output-only judge frequently confuses `done`, `partial`, `failed`, and `unclear` cases.

G Case Study: Recovery after a Failed Edit

We use one representative code-repair case to show how execution traces expose reliability differences that final pass rates alone hide. The task is to fix a sensitive-information leak in `tools/lark_auth.py`. In `device_flow_authorize()`, the original code interpolates the raw API message from `resp.get("msg")` into an exception when Device Flow initialization fails. Because this message may contain user access or refresh

tokens, the correct repair must route it through the existing `_sanitize_lark_error()` helper before raising the exception. The verifier requires the target file to exist, core symbols such as `device_flow_authorize`, `NeedsAuthError`, `get_user_token`, and `_sanitize_lark_error` to be preserved, the raw `{resp.get("msg")}` interpolation to disappear, and the Device Flow initialization failure path to still raise an error. Thus, the case requires a small but precise code change, not a high-level explanation, while preserving surrounding authentication logic.

Across repeated runs, 18 trajectories passed and 26 failed. Passing traces were not simply traces without tool errors. One GPT-5.5 trajectory first failed an exact edit because the supplied text did not match the file contents, receiving a `old text not found` error. The agent then re-read the file, located the actual surrounding code, rewrote the exception construction, ran the tests, and printed the required confirmation after all seven tests passed. This trajectory illustrates a successful recovery pattern: the first edit failed, but the agent used tool feedback to revise its plan and complete the task.

The failed trajectories show several distinct weaknesses. Some models made the sanitization change but omitted the required confirmation string, causing verifier failure even though the edit was mostly correct. Others changed exception text or control flow in a way that broke the expected Device Flow failure path. More severe failures occurred when agents tried to repair a truncated or partially read file and lost core functions such as `device_flow_authorize`. These runs often ended with plausible summaries claiming that the leak was fixed, while the resulting file no longer contained all required authentication symbols. Local-model runs also showed path-grounding and continuation failures, including reading from the wrong workspace path or stopping after partial tool actions. Table 12 summarizes these recovery and failure patterns. Overall, the case shows that failure is not always caused by missing domain knowledge. Many models understood that the raw API message should be sanitized, but success depended on execution reliability: recovering from edit mismatches, preserving nearby code, running tests, and satisfying the explicit completion contract. This is a realistic agentic failure mode in existing projects, where a small patch requires robust tool use, local state tracking, and post-edit verification.

Aspect	Observation
Task	Sanitize a raw Lark API error message before raising a Device Flow error
Target file	`tools/lark_auth.py`
Required behavior	Use `_sanitize_lark_error()` while preserving authentication logic
Verification	Static symbol checks, leak-pattern check, preserved raise path, test execution, and required stdout confirmation
Successful recovery	Failed exact edit, re-read file, applied corrected patch, ran tests, printed confirmation
Common failures	Missing confirmation, changed error path, lost core functions, wrong path, incomplete continuation
Capability tested	Tool recovery, precise code editing, state preservation, and verification discipline

Table 12: Recovery and failure patterns in the Lark authentication repair case. The case shows that success depends not only on knowing the required patch, but also on recovering from tool errors, preserving nearby code, and completing verification.

H Realness Metric Computation

Algorithm 1 specifies how the metrics in Table 1 are computed. It maps both the deployed reference stream and each benchmark to a shared taxonomy before computing JSD and TV. When comparable public prompts are available, it applies rule-based text-signal functions to compute the intent score and selected signal shares.

I Representative Task Cards

This appendix provides representative task cards. Each card summarizes the workspace setup, agent objective, expected artifacts, constraints, and oracle-grading signal for one task family. Table 13 shows a Manim educational video script workflow from the file-creation task type. The example illustrates how a raw session is converted into a standalone instruction, a bounded input workspace, and concrete verifier checks. It also shows that the released task exposes the necessary evidence without revealing the original private session. The verifier focuses on observable artifacts, such as file placement, syntax validity, required scene classes, configuration, and content signals, rather than subjective judgments of animation quality.

J Use of AI

We used large language models (LLMs) throughout the research and writing process as auxiliary tools for language polishing, grammar correction,

Algorithm 1 Computing Realness Metrics

Require: Deployed OpenClaw tool-use sessions R, benchmark artifacts $\{B_i\}$, shared taxonomy $\mathcal{C}$, and text-signal functions $\mathcal{F} = \{f_1, \ldots, f_8\}$

Ensure: JSD, TV, Intent score, and selected signal shares for each benchmark B_i

Map each real session in R to a category $c \in \mathcal{C}$
Count $n_R(c)$ and normalize $p_R(c) \leftarrow n_R(c) / \sum_{c' \in \mathcal{C}} n_R(c')$
for all benchmarks B_i **do**
    Map each task in B_i to a category $c \in \mathcal{C}$
    Count $n_{B_i}(c)$ and normalize $p_{B_i}(c) \leftarrow n_{B_i}(c) / \sum_{c' \in \mathcal{C}} n_{B_i}(c')$
    $m \leftarrow (p_R + p_{B_i})/2$
    $\mathrm{JSD}_i \leftarrow \frac{1}{2}\mathrm{KL}(p_R \| m) + \frac{1}{2}\mathrm{KL}(p_{B_i} \| m)$
    $\mathrm{TV}_i \leftarrow \frac{1}{2} \sum_{c \in \mathcal{C}} |p_R(c) - p_{B_i}(c)|$
    if B_i exposes comparable public task prompts **then**
        for all task prompts $x \in B_i$ **do**
            for $j = 1$ to 8 **do**
                Compute binary signal $f_j(x) \in \{0, 1\}$
            end for
        end for
        $\mathrm{Intent}_i \leftarrow |B_i|^{-1} \sum_{x \in B_i} \sum_{j=1}^{8} f_j(x)$
        $\mathrm{Share}_{i,j} \leftarrow |B_i|^{-1} \sum_{x \in B_i} f_j(x)$ for selected signals j
    else
        Mark text-realism metrics as unavailable
    end if
end for

and clarity improvement. In particular, LLM-based assistants were used to refine sentence structure, improve academic writing style, and identify potential grammatical inconsistencies in the manuscript.

In addition, our evaluation pipeline relies on API-based access to multiple commercial and open-source AI systems. We used these APIs to compare model behaviors and capabilities, and to assess the reliability, difficulty, and discriminative power of REALCLAWBENCH. These runs were integral to validating whether the benchmark distinguishes performance differences among diverse AI systems.

AI-assisted coding tools were also used during implementation and experimentation to help debug scripts, inspect runtime errors, analyze logs, and improve engineering efficiency. Furthermore, some figures and visual illustrations in the paper were generated or refined with the assistance of AI-based image generation and visualization tools to improve presentation quality and readability.

All AI-generated or AI-assisted content, including text revisions, code suggestions, evaluation runs, and figures, was carefully reviewed, independently verified, and edited by the authors before inclusion in the final manuscript.

Field	Description
Task ID	04212e3aa859be28
Subclass	A.file_create. Tier tier2_pastes_inline. Task tags [file_or_workspace, artifact]
Task	**Chain Rule Manim educational video script.** Create a Manim Community Edition v0.19.1 script for an educational video explaining the chain rule. The workspace already provides the storyboard file root/backprop-demo/chain_rule/storyboard.py. Refer to its scene descriptions, characters, and configuration information, but do not need to copy it exactly. **Output file path.** The script must be saved to the following path relative to the workspace root: gradient-visualization/gradient_chain/scenes.py. Any other location is considered non-compliant. **Required Scene classes.** Based on storyboard scenes S01, S04, S05, S06, S07, S08, and S10, create the corresponding Manim Scene classes. Each class must directly inherit from Scene. Class names must use the format S{ID}_{short_desc}. 1. S01_SimpleNetwork: corresponds to storyboard scene S01. Show a simple multi-layer neural network, with data flowing from left to right and a large X displayed at the output. 2. S04_DominoToNetwork: corresponds to S04. Domino tiles transform into neural-network layers, with at least one transformation animation. 3. S05_ChainRuleFormula: corresponds to S05. Animate the chain-rule formula dy/dx = dy/du · du/dx, rendered using MathTex or Tex. 4. S06_FormulaWithNetwork: corresponds to S06. Overlay the chain-rule formula on the neural-network diagram so that the formula and the network are visible at the same time. 5. S07_ConcreteExample: corresponds to S07. Show a concrete composite-function derivative example, such as the differentiation process for (3x+1)^4. 6. S08_ComputeGradient: corresponds to S08. Use the chain rule to compute the gradient of a certain layer and reflect the idea of error backpropagation. 7. S10_GradientDescentStep: corresponds to S10. Demonstrate one gradient-descent update step and show how parameters are updated according to the chain-rule gradient. If a storyboard scene does not contain enough content to form a complete animation, the agent may extend it appropriately, but the class names and scene themes must still match. **Global configuration.** In the global scope of the script, set Manim configuration to match VIDEO_CONFIG in storyboard.py: config.pixel_width = 1280, config.pixel_height = 720, and config.background_color = WHITE or "#FFFFFF". **Content requirements.** The script must contain Chinese characters, such as character dialogue or explanatory text. It must use MathTex or Tex at least once to render a mathematical formula. It is recommended, but not required, to use colors such as #E53935, #1E88E5, and #43A047 to highlight key elements.
Input workspace	Files placed in the workspace before the agent runs: • gradient-visualization/README.md (231 B) • gradient-visualization/gradient_chain/init.py (72 B) • root/backprop-demo/chain_rule/README.md (267 B) • root/backprop-demo/chain_rule/storyboard.py (3807 B)
Rubric	Must pass: 1. file_exists — gradient-visualization/gradient_chain/scenes.py exists and is nonempty. 2. syntax_valid — the file can be compiled with py_compile. 3. manim_import — the file contains from manim import * or an equivalent Manim import. 4. scene_classes_defined — at least six of the seven required Scene classes are defined, and these classes inherit from Scene. 5. config_set — pixel_width=1280 and pixel_height=720 are set, and a WHITE background is used. 6. content_richness — the file contains Chinese characters and uses MathTex or Tex at least once. 7. has_main_block — the bottom of the file contains an if __name__ == "__main__": block.

Table 13: Representative task card showing the released instruction, workspace evidence, and verifier checks. The example illustrates how a private session is projected into a standalone, bounded, and automatically graded benchmark instance.